Teacher's Methodology Guide

Understanding How Your Teacher Solves Uninformed Search Exercises

Based on Lecture 2 Solutions

Decoding the Problem-Solving Patterns

Contents

1 Introduction: Decoding the Teacher's Approach **2**

2 Pattern 1: State Representation Strategies **2**

2.1 The "In Vehicle/Container" Pattern 2

2.2 The "Visited/Completion Tracking" Pattern 3

2.3 The "Team Assignment" Pattern 4

3 Pattern 2: Action Design and Applicability **5**

3.1 Encoding Constraints in Action Applicability 5

3.2 Capacity Constraints 6

4 Pattern 3: UCS Execution Methodology **7**

4.1 The Teacher's Table Format 7

4.2 Tie-Breaking Convention 8

4.3 Step-by-Step Execution Pattern 9

5 Pattern 4: IDS Execution Methodology **10**

5.1 The Teacher's IDS Table Format 10

5.2 Ordering of Successors 10

6 Common Conventions and Hidden Assumptions **13**

6.1 Cost Conventions 13

6.2 Goal Test Conventions 13

6.3 Initial State Conventions 14

7 Problem-Specific Insights from Teacher's Solutions **15**

7.1 Exercise 1.1 - State Space Analysis 15

7.2 Exercise 1.2 - Team Assignment 16

7.3 Exercise 1.3 - Meeting Point 16

8 Exam Strategy Based on Teacher's Patterns **17**

8.1 Reading the Problem Like the Teacher 17
8.2 Execution Strategy 18
8.3 Common Mistakes to Avoid 19

9 Quick Reference: Teacher's Notation Guide **20**

10 Final Checklist: Solving Like Your Teacher **21**

1 Introduction: Decoding the Teacher's Approach

Purpose of This Guide

This guide extracts the **methodology and patterns** from your teacher's solutions to help you understand:

- How the teacher represents states in different problem types
- Common patterns used across multiple exercises
- The systematic approach to execution (UCS, IDS)
- Hidden assumptions and conventions (like lexicographic tie-breaking)
- How to track and organize your work the way the teacher expects

Use this guide to: Bridge the gap between knowing the algorithms and solving exercises the way your teacher solves them.

2 Pattern 1: State Representation Strategies

2.1 The "In Vehicle/Container" Pattern

Pattern: Objects That Can Be Inside Other Objects

When passengers can be in a taxi, people in an elevator, etc., the teacher uses a **special symbol** to denote "inside":
Teacher's Convention:

- Use symbol t (italic) for "in taxi"
- Use symbol e for "in elevator"
- DON'T reuse location names to mean "inside"

Example from Exercise 4.1 (Taxi):

$$s = (t, p_1, p_2) \text{ where } t, p_i \in \{A, B, C, D, E, F, t\}$$

- If $p_1 = C$: passenger 1 is at location C
- If $p_1 = t$: passenger 1 is **in the taxi**
- Taxi location is $t \in \{A, B, C, D, E, F\}$ (one of the map locations)

Why this works:

- Clear distinction between being at a location and being inside
- Actions like pickup: $(C, C, E) \to (C, t, E)$ are clean and unambiguous
- Dropoff: $(F, t, E) \to (F, F, E)$ clearly shows passenger now at F

Teacher's Solutions Using This Pattern

Exercise 1.4 - Elevator Scheduling:

$$s = (e, c, p_1, p_2, p_3) \text{ where } p_i \in \{1, \ldots, F\} \cup \{e\}$$

- e = elevator floor
- c = current capacity (number of passengers)
- $p_i \in \{1, 2, 3\}$ means passenger at that floor
- $p_i = e$ means passenger in elevator

Exercise 4.1 - Taxi Problem:

$$s = (t, p_1, p_2) \text{ where } p_i \in \{A, B, C, D, E, F, t\}$$

- Initial: (A, C, E) - taxi at A, passengers at C and E
- After pickup: (C, t, E) - passenger 1 now in taxi
- After delivery: (F, F, E) - passenger 1 delivered to F

2.2 The "Visited/Completion Tracking" Pattern

Pattern: Binary Flags for Completion

When the goal requires "all X visited/completed", the teacher adds **binary state variables** that flip from 0 to 1:
Teacher's Convention:

- Add variables for each thing that needs to be visited/completed
- Use 0 = not yet visited, 1 = visited
- Update these when entering/completing
- Goal: all flags = 1 (plus any other constraints)

Exercise 4.2 - Museum Guard Patrol

State Representation:

$$s = (g_1, g_2, A, B, C, D, X)$$

- $g_1, g_2 \in \{A, B, C, D, X\}$ - guard locations
- $A, B, C, D, X \in \{0, 1\}$ - binary flags for "room visited"

Initial State: $(X, B, 0, 1, 0, 0, 1)$

- Guard 1 at X, Guard 2 at B
- Rooms B and X already visited (guards start there)
- Rooms A, C, D not yet visited

Goal States: $(*, *, 1, 1, 1, 1, 1)$ for any g_1, g_2

- All rooms must be visited
- Don't care where guards end up

Action Effect Example:

- Action: $m1$-X-A (move guard 1 from X to A)
- Before: $(X, B, 0, 1, 0, 0, 1)$
- After: $(A, B, 1, 1, 0, 0, 1)$ - notice A flag flipped to 1!

2.3 The "Team Assignment" Pattern

Pattern: Binary Assignment

When assigning items to groups/teams, use **binary vectors**:
From Exercise 1.2 - Team Assignment:

$$s = (A, B, C, D, E, F) \text{ where each variable} \in \{0, 1\}$$

- $0 =$ belongs to team 1
- $1 =$ belongs to team 2
- State $(0, 1, 1, 1, 0, 0)$ means: A,E,F in team 1; B,C,D in team 2

Actions:

- X-to-0: move engineer X to team 1 (set $X = 0$)
- X-to-1: move engineer X to team 2 (set $X = 1$)
- Applicable only if engineer currently in other team

Goal: Check if team skill totals are balanced (difference ≤ 2 for each skill)

3 Pattern 2: Action Design and Applicability

3.1 Encoding Constraints in Action Applicability

Teacher's Philosophy

Key Insight: Constraints are NOT separate checks - they are built into **when actions can be applied**.
Don't think: "Can I do this action? Let me check constraints separately."
Think: "The action definition includes all the conditions for when it's applicable."

Exercise 4.2 - Permanent Guard Constraint

Problem Constraint: Room X must be **permanently guarded** - at least one guard must always be at X.
Teacher's Encoding:
Actions moving FROM room X require BOTH guards to be at X:

- Action: $m1$-X-A (move guard 1 from X to A)
- **Applicability:** Can only apply if $(g_1 = X)$ AND $(g_2 = X)$
- **Why?** When guard 1 leaves, guard 2 stays behind - X remains guarded!

Execution Pattern from Teacher's Solution:

Step	State	Action	Key Observation
1	$(X, B, 0, 1, 0, 0, 1)$	$m2$-B-A	Only g2 can move (g1 at X)
2	$(X, A, 1, 1, 0, 0, 1)$	$m2$-A-X	g2 joins g1 at X
3	$(X, X, 1, 1, 0, 0, 1)$	$m1$-X-A	**NOW** g1 can leave X!

Teacher's Pattern: First gather both guards at X, THEN split them to explore.

3.2 Capacity Constraints

Pattern: Checking Capacity Before Boarding

From Exercise 1.4 - Elevator (capacity = 2):
Pickup actions check TWO conditions:

1. Passenger and elevator at same floor
2. Capacity allows: $c < C$ (current capacity < max capacity)

Teacher's approach:

- Track capacity explicitly: $c \in \{0, 1, 2\}$ in state
- $board\text{-}p_i\text{-}f_j$ applicable iff:
  - $e = f_j$ (elevator at floor j)
  - $p_i = f_j$ (passenger i at floor j)
  - $c < 2$ (capacity available)
- Action effect: $c := c + 1$ and $p_i := e$

Alternative pattern from Taxi (capacity = 1):

- Don't track capacity explicitly
- Implicit check: can't pickup p_1 if $p_2 = t$ (other passenger in taxi)
- Pickup applicable iff $p_i \neq t$ (passenger not already in) AND $p_j \neq t$ for $j \neq i$ (other passenger not in)

4 Pattern 3: UCS Execution Methodology

4.1 The Teacher's Table Format

Standard UCS Execution Table

Teacher's preferred format from Exercise 4.1 and 4.2:

Step	State	g-value
1	(initial state)	0
2	...	...

What the teacher tracks:

- Step number (sequential)
- State being expanded (not node - just the state tuple)
- g-value of that state
- Sometimes: Open list contents after expansion

What the teacher does NOT show explicitly:

- Closed set in table (implied: all previous states)
- Goal test column (mentioned in text only when goal found)
- Actions applied (shown in explanation, not table)

4.2 Tie-Breaking Convention

Teacher's Tie-Breaking Rule

From Exercise 4.1: "When deciding which state to pop from the open list, break ties in favour of the **lexicographically smaller** state encoding."
What this means:

1. Compare states component by component (left to right)
2. Use alphabetical order for locations
3. Symbol t (in taxi) orders AFTER regular locations
4. Numbers order naturally (1 ¡ 2 ¡ 3)

Examples from teacher's solutions:

- (C, C, E) comes BEFORE (C, t, E) because $C < t$ in second position
- (C, t, E) comes BEFORE (D, C, E) because $C < D$ in first position
- $(e = C, p_1 = C, p_2 = E)$ before $(e = C, p_1 = C, p_2 = t)$
- $(e = C, p_1 = F, p_2 = E)$ before $(e = A, p_1 = C, p_2 = E)$ - wait, this checks second component first when first is same!

Correct interpretation: Lexicographic = dictionary order on the ENTIRE tuple:

```
(A,B,C) < (A,B,D)  -- third component breaks tie
(A,C,E) < (A,t,E)  -- second component: C < t
(B,C,E) < (C,C,E)  -- first component decides
```

4.3 Step-by-Step Execution Pattern

Teacher's UCS Execution - Exercise 4.1 (First 5 Steps)

Problem: Taxi at A, passengers at C and E, goal: get ONE passenger to F
Step 1: Expand (A, C, E), $g = 0$

- Actions: $drive_{AB}$ (cost 1), $drive_{AD}$ (cost 2)
- Successors: $(B, C, E) : 1$, $(D, C, E) : 2$
- Both new $\Rightarrow$ insert into open

Step 2: Expand (B, C, E), $g = 1$ (minimum in open)

- Actions: $drive_{BC}$ (cost 1)
- Successor: $(C, C, E) : 2$
- New $\Rightarrow$ insert
- Open: $(C, C, E) : 2$, $(D, C, E) : 2$ (tie!)

Step 3: Expand (C, C, E), $g = 2$ (tie-break: $C < D$ lexicographically)

- Actions: $drive_{CF}$ (cost 2), $pickup_1$ (cost 0) - **taxi and p1 both at C!**
- Successors: $(F, C, E) : 4$, $(C, t, E) : 2$
- Both new $\Rightarrow$ insert
- Open: $(C, t, E) : 2$, $(D, C, E) : 2$, $(F, C, E) : 4$

Step 4: Expand (C, t, E), $g = 2$ (tie-break: $(C, t, E) < (D, C, E)$)

- **Passenger 1 now in taxi!**
- Actions: $drive_{CB}$ (cost 1), $drive_{CF}$ (cost 2)
- Successors: $(B, t, E) : 3$, $(F, t, E) : 4$ - **can deliver p1 to F!**
- Cannot pickup p2 (taxi at C, p2 at E)

Step 5: Expand (D, C, E), $g = 2$

- Actions: $drive_{DE}$ (cost 1)
- Successor: $(E, C, E) : 3$ - **taxi now at p2's location!**

Teacher's Pattern:

- Always expand minimum g-value
- Show state clearly at each step
- Highlight key events (pickup, at goal location, etc.)
- Track what's now possible after state changes

5 Pattern 4: IDS Execution Methodology

5.1 The Teacher's IDS Table Format

IDS Execution Table Structure

From Exercises 5.1 and 5.2:
The teacher uses a **two-column format** showing:

1. **Depth limit** for current iteration
2. **Step within iteration** - sequential counter
3. **State visited**

Example format:

Depth Limit	Step	State
0	1	(A, C, E)
1	1	(A, C, E)
1	2	(B, C, E)
1	3	(D, C, E)
2	1	(A, C, E)
2	2	(B, C, E)
2	3	(C, C, E)
2	4	(D, C, E)

5.2 Ordering of Successors

Lexicographic Ordering in IDS

From Exercise 5.1 and 5.2 instructions: "When expanding a state, order successors **lexicographically**."
Teacher's pattern from solutions:

- When expanding (A, C, E), generate (B, C, E) and (D, C, E)
- Visit in order: (B, C, E) first (because $B < D$), then (D, C, E)
- For museum guard: $(X, A, \ldots) < (X, B, \ldots) < (X, C, \ldots)$

Result: DFS exploration follows alphabetical order at each branch

Teacher's IDS Execution - Exercise 5.1 (Taxi)

Depth 0:

- Visit: (A, C, E) - cannot expand, return CUTOFF

Depth 1:

- Visit: (A, C, E) - expand (depth $0 < 1$)
- Generate successors: (B, C, E), (D, C, E) (alphabetical order)
- Visit: (B, C, E) - depth 1, cannot expand, CUTOFF
- Visit: (D, C, E) - depth 1, cannot expand, CUTOFF
- Overall: CUTOFF

Depth 2:

- Visit: (A, C, E) - expand
- Visit: (B, C, E) - expand (depth $1 < 2$)
- Visit: (C, C, E) - depth 2, CUTOFF
- Backtrack to (A, C, E)
- Visit: (D, C, E) - expand (depth $1 < 2$)
- Visit: (E, C, E) - depth 2, CUTOFF
- Overall: CUTOFF

Teacher's pattern:

- Show progression through depth limits
- List states in order they are VISITED (DFS order)
- Mark when cutoff happens
- Continue until goal found

Teacher's IDS Execution - Exercise 5.2 (Museum Guard)

Depth 1:

Step	g_1	g_2	**A**	**B**	**C**	**D,X**
1	X	B	0	1	0	0,1
2	X	A	1	1	0	0,1
3	X	C	0	1	1	0,1
4	X	X	0	1	0	0,1

Key observation: Only guard 2 can move initially (guard 1 at X, needs both to leave X)
Depth 2:

- Expand $(X, B, 0, 1, 0, 0, 1)$
- Then expand $(X, A, 1, 1, 0, 0, 1)$ (depth 1)
- Visit: $(X, B, 1, 1, 0, 0, 1)$ - depth 2, CUTOFF
- Visit: $(X, X, 1, 1, 0, 0, 1)$ - depth 2, CUTOFF
- Backtrack and continue...

Teacher's convention: When showing states in table, abbreviate D and X columns if space limited

6 Common Conventions and Hidden Assumptions

6.1 Cost Conventions

Teacher's Cost Assignments

Default cost assumptions:

Action Type	Typical Cost	Example
Movement	Distance/time	$drive_{AB}$ costs edge weight
Pickup/Board	0 or 1	Exercise 4.1: cost = 1 for pickup
Dropoff/Exit	0 or 1	Usually 0 unless stated
State change	0	Team switching costs 0

From Exercise 4.1 - Taxi:

- $drive_{XY}$: cost = distance from X to Y (map edges)
- $getin_iX$: cost = 1 (problem statement says "cost is 1")
- $getout_iX$: cost = 1

From Exercise 1.4 - Elevator:

- Movement actions: cost = 1 (one floor of travel)
- Board/exit actions: cost = 0 (minimize distance, not boarding time)

Teacher's principle: If problem says "minimize travel distance", only movement costs; other actions cost 0.

6.2 Goal Test Conventions

How Teacher Writes Goal Conditions

Pattern 1: Partial assignment (don't care about some variables)

- Taxi: $(*, F, F)$ - any taxi location, both passengers at F
- Museum: $(*, *, 1, 1, 1, 1, 1)$ - any guard positions, all rooms visited

Pattern 2: Disjunction (OR conditions)

- Taxi: $(*, F, *) \vee (*, *, F)$ - **at least one** passenger at F
- Can also write: $(p_1 = F) \vee (p_2 = F)$

Pattern 3: Conjunction (AND conditions)

- Meeting Point: (a, e, f) where $a = e = f$ - all friends at same location
- Team Assignment: skill differences ≤ 2 AND exactly 3 per team

Teacher's style: Use mathematical notation $(\vee, \wedge)$ or natural language, whichever is clearer.

6.3 Initial State Conventions

How Teacher Specifies Initial States

Pattern from solutions:
Exercise 4.1 - Taxi:

- Text: "Taxi at A, passengers at C and E"
- State: $I = (A, C, E)$

Exercise 4.2 - Museum Guard:

- Text: "Guards start at X and B"
- State: $I = (g_1 = X, g_2 = B, A = 0, B = 1, C = 0, D = 0, X = 1)$
- **Key detail:** Rooms where guards START are marked visited (B=1, X=1)!

Exercise 1.4 - Elevator:

- Text: "Elevator on floor 1, passengers at their source floors"
- State: $I = (e = 1, p_1 = 1, p_2 = 1, p_3 = 3)$
- Capacity $c = 0$ (empty initially)

Teacher's pattern:

- Initialize "visited" flags based on starting positions
- Capacity starts at 0 (empty)
- All variables must have concrete values (no wildcards in initial state)

7 Problem-Specific Insights from Teacher's Solutions

7.1 Exercise 1.1 - State Space Analysis

Teacher's Definitions

Successors of C: A, K, G

- Read from arrows LEAVING C
- Follow the action arrows to destination states

Predecessors of E: J, F

- Read from arrows ENTERING E
- States from which you can reach E in one action

Solvable states: A, B, C, E, F, G, H, I, J, K

- Can reach a goal (F or H) from these states
- Teacher's method: trace backwards from goals

Dead-ends: D

- No actions available (no outgoing arrows)
- Cannot reach goal from here

Not reachable from B: A, C, D, G, H, I, K

- Cannot reach these starting from B
- Teacher's method: trace forward from B

Teacher's approach: Draw/trace on the graph, don't try to compute in your head.

7.2 Exercise 1.2 - Team Assignment

Teacher's Encoding Choices

State: (A, B, C, D, E, F) where each $\in \{0, 1\}$
Why binary vector?

- Simple: each engineer either in team 1 or team 2
- Easy to compute skill totals: sum values for each team
- Actions are symmetric: flip any variable

Actions:

- X-to-0: applicable iff $X = 1$, changes to $X = 0$
- X-to-1: applicable iff $X = 0$, changes to $X = 1$
- Cost $= 0$ (reassignment is free)

Goal test:

- Exactly 3 in each team: $\sum X_i = 3$
- Balanced skills: for each skill s, $|\text{team1}_s - \text{team2}_s| \leq 2$

Teacher's note: "The solution is just the goal state here. The path itself is useless for us."

- Don't care HOW we rearranged people, just that we found balanced teams
- This is a **configuration problem**, not a **path problem**

7.3 Exercise 1.3 - Meeting Point

Teacher's State Design

State: (a, e, f) - locations of Alice, Emil, Frank
Actions:

- *move-a-C-E*: move Alice from C to E
- Cost = edge weight for that move
- Each person can move independently

Goal: (L, L, L) for any location L (all at same place)
Easy to enumerate: (A, A, A), (B, B, B), (C, C, C), (D, D, D), (E, E, E)
Teacher's note: "The solution here gives us not only the meeting point, but also a route for each friend"

- **This IS a path problem** - we care about the path!
- From solution path, extract each person's route:
  - Alice's route: follow changes to a
  - Emil's route: follow changes to e
  - Frank's route: follow changes to f

8 Exam Strategy Based on Teacher's Patterns

8.1 Reading the Problem Like the Teacher

Teacher's Problem Analysis Checklist

When you see a new problem, ask in this order:

1. **What objects/agents can move or change?**
   - Taxi, elevator, people, guards, teams
   - These become state variables
2. **Is there a "container" relationship?**
   - Passengers in taxi, people in elevator
   - Use the "special symbol" pattern (t, e)
3. **Do I need to track completion/visited?**
   - "All rooms visited", "all passengers delivered"
   - Add binary flags for each thing to track
4. **Are there permanent constraints?**
   - "Room must always be guarded", "capacity limit"
   - Encode in action applicability, not separate checks
5. **What costs?**
   - Movement usually costs (distance/time)
   - State changes usually cost 0
   - Unless problem says otherwise!
6. **Is this a path problem or configuration problem?**
   - Path: care about HOW we got there (routes, schedules)
   - Configuration: care only about final state (team assignments)

8.2 Execution Strategy

How to Execute Like the Teacher

For UCS:

1. Set up simple table: Step — State — g-value
2. Always expand minimum g-value
3. For ties, use lexicographic order:
   - Compare tuples left-to-right
   - $A < B < C < \ldots < t$ (special symbols last)
   - $(A, B, C) < (A, B, D) < (A, C, A) < (B, A, A)$
4. Track what's new/important: "passenger now in taxi!", "at goal location!"
5. Continue until goal expanded

For IDS:

1. Use depth limit — step — state format
2. Generate successors in lexicographic order
3. Visit in DFS order (depth-first within each limit)
4. Show progression through depth limits
5. Mark cutoff when depth limit hit

General tips:

- Write clearly: use tuples like (A, C, E), not prose
- Show your work: list actions, show successors
- Highlight key events: pickups, goals reached, constraints satisfied
- Check your arithmetic: verify g-values are cumulative

8.3 Common Mistakes to Avoid

What NOT to Do (Based on Teacher's Patterns)

1. **Wrong state representation:**
   - Including constants (passenger destinations) in state
   - Forgetting critical variables (capacity, visited flags)
   - Confusing "at location" with "inside container"
2. **Wrong action applicability:**
   - Pickup without checking if at same location
   - Ignoring capacity constraints
   - Forgetting permanent constraints (guard always at X)
3. **Wrong costs:**
   - Making everything cost 1 (movement should cost distance!)
   - Forgetting pickup costs if problem says so
   - Using cost 1 when problem wants to minimize distance (should be edge weights)
4. **Execution errors:**
   - Not using lexicographic tie-breaking
   - Forgetting to mark rooms as visited when entering
   - Testing goal when generating instead of when expanding
   - Not tracking g-values cumulatively
5. **Presentation errors:**
   - Messy notation: write (A, C, E) not "taxi at A and passengers at C and E"
   - Missing steps in execution
   - Not showing what actions were applied
   - Forgetting to answer the question (show first 5 steps vs. full solution)

9 Quick Reference: Teacher's Notation Guide

Standard Notation from Teacher's Solutions

State Representation:

- Tuple: $s = (v_1, v_2, \ldots, v_n)$
- Example: (A, C, E) for taxi problem
- Use italics t, e for "in container"
- Use 0/1 for binary flags

Actions:

- Movement: $drive_{XY}$, $move\text{-}a\text{-}C\text{-}E$
- Container: $pickup_i$, $dropoff_i$, $board\text{-}p_i\text{-}f_j$, $exit\text{-}p_i\text{-}f_j$
- Team: X-to-0, X-to-1
- Guard: $m1\text{-}X\text{-}A$ (move guard 1 from X to A)

Transitions:

- Notation: (s, a, s') or $s \xrightarrow{a} s'$
- Example: $(A, C, E) \xrightarrow{drive_{AB}} (B, C, E)$

Goal Conditions:

- Wildcards: $(*, F, F)$ means "any value for first component"
- OR: $(p_1 = F) \vee (p_2 = F)$
- AND: $(a = e = f) \wedge (\text{all visited} = 1)$

Sets:

- Open: list of (state, g-value) pairs
- Closed: set of states (no g-values)
- Actions: $\{drive_{AB}, drive_{AD}, pickup_1\}$

Tables:

- UCS: Step — State — g-value
- IDS: Depth limit — Step — State
- Sometimes add: Actions — Successors — Notes

10 Final Checklist: Solving Like Your Teacher

The Teacher's Mental Model

Before you start writing:

1. Identify the pattern (container? visited tracking? configuration?)
2. Choose state representation (use teacher's conventions)
3. List actions with clear applicability rules
4. Determine costs (movement vs. state changes)
5. Write goal condition precisely

During execution:

1. Use teacher's table format
2. Apply lexicographic tie-breaking
3. Show states clearly as tuples
4. Highlight important events
5. Verify g-values are cumulative

After solving:

1. Check: did I answer what was asked? (first N steps vs. full solution)
2. Check: are all constraints satisfied?
3. Check: is notation consistent?
4. Check: is the solution clear and well-organized?

Remember: Your teacher values **clarity and systematic approach** over speed. Show your work, use consistent notation, and follow the patterns from the solutions.

Key Takeaway

The teacher's solutions reveal consistent patterns:
special symbols for containers, **binary flags for tracking**,
constraints in applicability, and **lexicographic tie-breaking**.

Master these patterns and you'll solve exercises the way your teacher expects!